High Level Overview

Llama (meta.ai) has trouble reading the provided file, and seems to have memory closer to a lot older versions of ChatGPT (early 2023) which do not remember what was just talked about. For some of the questions with several parts, it doesn't even get to answering each subpart individually. It also formats answers without latex, and requires specific prompts explicitly detailing that responses should be formatted with latex to make it easy for me to read. With how many parameters Llama is supposed to be trained on, this was rather disappointing.

It was able to answer the first question completely correctly, and the second one with only two subparts incorrect. After that, work was mostly hallucinations as it wasn't able to answer questions with multiple subparts while maintaining consistency.

Complete Log with comments

Prompt: solve question 2 (with file attached)
Response: Let's go through each part of question 2 in the Vector Calculus Review section.
Question 2:
Given the function f(x) = x^T c, where x is a vector and c is a constant vector, we need to find the derivative of f(x) with respect to x.
Part (a): Show ∂/∂x (x^T c) = c^T
To find the derivative, we can use the definition of the derivative:
∂/∂x (x^T c) = ∂/∂x (c^T x) since x^T c = c^T x
Using the product rule, we get:
∂/∂x (c^T x) = c^T
So, ∂/∂x (x^T c) = c^T
Part (b): Show ∂/∂x (x^T x) = 2x^T
Let's find the derivative:
∂/∂x (x^T x) = ∂/∂x (||x||^2)
Using the chain rule, we get:
∂/∂x (||x||^2) = 2x^T
Part (c): Show ∂/∂x (x^T Ax) = x^T (A + A^T)
Let's find the derivative:
∂/∂x (x^T Ax) = ∂/∂x (x^T A x)
Using the product rule, we get:
∂/∂x (x^T A x) = x^T A + x^T A^T
= x^T (A + A^T)
Part (d): Show ∂/∂x (b^T Ax) = b^T A
Let's find the derivative:
∂/∂x (b^T Ax) = ∂/∂x (b^T A x)
Using the product rule, we get:

∂/∂x (b^T A x) = b^T A
Part (e): Show ∂/∂x (x^T Ax) = (A + A^T)x when A is symmetric
If A is symmetric, then A = A^T.
∂/∂x (x^T Ax) = x^T (A + A^T)
= x^T (A + A)
= x^T (2A)
= 2x^T A
= (A + A^T)x
Let me know if you have any questions or need further clarification on any of these parts.
Also, note that these are standard results in matrix calculus, and you can find more information on these topics in resources like the Matrix Cookbook or online lecture notes.
My response: hm the explanations are little hard to read are you unable to use latex display? also the explanations are a bit unclear
Llama response:

Question 2: Vector Calculus Review

Part (a): Show $\frac{\partial}{\partial x}(x^T c) = c^T$

To find the derivative of $x^T c$ with respect to x, note that $x^T c = c^T x = \sum_{i=1}^{n} c_i x_i$.

The derivative with respect to x_j is c_j, so the derivative with respect to x is c.

Thus, $\frac{\partial}{\partial x}(x^T c) = c^T$.

Part (b): Show $\frac{\partial}{\partial x}(x^T x) = 2x^T$

Note that $x^T x = \sum_{i=1}^{n} x_i^2$.

The derivative with respect to x_j is $2x_j$, so the derivative with respect to x is $2x^T$.

Thus, $\boxed{\frac{\partial}{\partial x}(x^T x) = 2x^T}$.

Part (c): Show $\frac{\partial}{\partial x}(x^T A x) = x^T(A + A^T)$

Let's expand $x^T A x = \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ij} x_i x_j$.

The derivative with respect to x_k is $\sum_{j=1}^{n} A_{kj} x_j + \sum_{i=1}^{n} A_{ik} x_i = (Ax)_k + (A^T x)_k$.

Thus, $\frac{\partial}{\partial x}(x^T A x) = x^T A + x^T A^T = x^T(A + A^T)$.

Part (d): Show $\frac{\partial}{\partial x}(b^T A x) = b^T A$

Note that $b^T A x = \sum_{i=1}^{n} \sum_{j=1}^{n} b_i A_{ij} x_j$.

The derivative with respect to x_k is $\sum_{i=1}^{n} b_i A_{ik} = (b^T A)_k$.

Thus, $\frac{\partial}{\partial x}(b^T A x) = b^T A$.

Part (e): Show $\frac{\partial}{\partial x}(x^T A x) = (A + A^T)x$ when A is symmetric

If A is symmetric, then $A = A^T$.

From part (c), we have $\frac{\partial}{\partial x}(x^T A x) = x^T(A + A^T) = x^T(A + A) = x^T(2A) = 2x^T A$.

Since $(A + A^T)x = (A + A)x = 2Ax$, we have $\frac{\partial}{\partial x}(x^T A x) = (A + A^T)x = 2Ax$.

I hope this clears up any confusion. Let me know if you have further questions or need more clarification.

Prompt: Now do question 3

Llama response:

It seems like you're looking for help with question 3, "Least Squares and the Min-norm problem from the Perspective of SVD".

Given the SVD of X = UΣV^T, where U and V are orthonormal square matrices, and Σ is an m x n matrix with singular values σi along the "diagonal", let's break down the problem:

Part (a): Overdetermined System (m > n)

To find the weights w that minimize the error between Xw and y, we want to solve:

min_w ||Xw - y||^2
The solution is given by:
w = (X^T X)^-1 X^T y
Part (b): Plug in SVD and Simplify
Substituting X = UΣV^T, we get:
w = (V Σ^T U^T U Σ V^T)^-1 V Σ^T U^T y
= (V Σ^T Σ V^T)^-1 V Σ^T U^T y
= V (Σ^T Σ)^-1 V^T V Σ^T U^T y
= V (Σ^T Σ)^-1 Σ^T U^T y
Part (c): Left-Inverse
If we left-multiply X by the matrix A = (X^T X)^-1 X^T, we get:
X A = X (X^T X)^-1 X^T
= U Σ V^T (V Σ^T U^T U Σ V^T)^-1 V Σ^T U^T
= U Σ V^T V (Σ^T Σ)^-1 V^T V Σ^T U^T
= U Σ (Σ^T Σ)^-1 Σ^T U^T
= U U^T
Since U is orthonormal, U U^T is a projection matrix onto the column space of U.
Part (d): Underdetermined System (m < n)
To find the minimum norm solution, we want to solve:
min ||w||^2 s.t. Xw = y
The solution is given by:
w = X^T (X X^T)^-1 y
Part (e): Plug in SVD and Simplify
Substituting X = U Σ V^T, we get:
w = V Σ^T U^T (U Σ V^T V Σ^T U^T)^-1 y
= V Σ^T U^T (U Σ Σ^T U^T)^-1 y
= V Σ^T U^T U (Σ Σ^T)^-1 U^T y
= V Σ^T (Σ Σ^T)^-1 U^T y
Part (f): Right-Inverse
If we right-multiply X by the matrix B = X^T (X X^T)^-1, we get:
X B = X X^T (X X^T)^-1
= U Σ V^T V Σ^T U^T (U Σ Σ^T U^T)^-1
= U Σ Σ^T U^T U (Σ Σ^T)^-1 U^T
= U U^T
Since U is orthonormal, U U^T is a projection matrix onto the column space of U.
Let me know if you have any questions or need further clarification on any of these parts.
My response: again use latex notation

Llama responded with clearer formatting but the same content

Let's break down the problem of Least Squares and the Min-norm problem from the Perspective of SVD.

Part (a): Overdetermined System ($m > n$)

To find the weights $\mathbf{w}$ that minimize the error between $\mathbf{X}\mathbf{w}$ and $\mathbf{y}$, we want to solve:

$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

The solution is given by:

$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Part (b): Plug in SVD and Simplify

Substituting $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, we get:

$\mathbf{w} = (\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^{-1}\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T\mathbf{y}$

$= \mathbf{V}(\mathbf{\Sigma}^T\mathbf{\Sigma})^{-1}\mathbf{V}^T\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T\mathbf{y}$

$= \mathbf{V}(\mathbf{\Sigma}^T\mathbf{\Sigma})^{-1}\mathbf{\Sigma}^T\mathbf{U}^T\mathbf{y}$

Part (c): Left-Inverse

If we left-multiply $\mathbf{X}$ by the matrix $\mathbf{A} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$, we get:

$\mathbf{X}\mathbf{A} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$

$= \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T(\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^{-1}\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T$

$= \mathbf{U}\mathbf{U}^T$

Since $\mathbf{U}$ is orthonormal, $\mathbf{U}\mathbf{U}^T$ is a projection matrix onto the column space of $\mathbf{U}$.

Part (d): Underdetermined System ($m < n$)

To find the minimum norm solution, we want to solve:

$\min \|\mathbf{w}\|^2 \text{ s.t. } \mathbf{X}\mathbf{w} = \mathbf{y}$

The solution is given by:

$\mathbf{w} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{y}$

My response: for part c, left multiplication should be computing AX not XA

For part (c), we need to compute AX:

AX = (X^T X)^-1 X^T X

= (V Σ^T U^T U Σ V^T)^-1 V Σ^T U^T U Σ V^T

= (V Σ^T Σ V^T)^-1 V Σ^T Σ V^T

= V (Σ^T Σ)^-1 V^T V Σ^T Σ V^T

= V (Σ^T Σ)^-1 Σ^T Σ V^T

= V V^T

Since V is orthonormal, V V^T = I, so AX = I.

This shows that A is a left-inverse of X. Let me know if you have any further questions!

(It got it right now)

Followup from me: similarly for part f compute BX not XB

For part (f), we need to compute BX:

BX = X^T (X X^T)^-1 X

= V Σ^T U^T (U Σ V^T V Σ^T U^T)^-1 U Σ V^T

= V Σ^T U^T (U Σ Σ^T U^T)^-1 U Σ V^T

= V Σ^T U^T U (Σ Σ^T)^-1 U^T U Σ V^T

= V Σ^T (Σ Σ^T)^-1 Σ V^T

Since Σ^T (Σ Σ^T)^-1 Σ is a projection matrix onto the row space of Σ (or column space of Σ^T), and V is orthonormal, we can simplify this expression.

However, given the properties of the Moore-Penrose pseudoinverse, we know that X^T (X X^T)^-1 X is the projection matrix onto the row space of X, which in this case, given X = U Σ V^T, would project onto the row space spanned by V.

Given the specific structure of Σ and properties of V and U, we see that:

BX = V V^T

If X has full row rank (which would be the case when m < n and X has rank m), then V V^T would represent the projection onto the entire row space of X if all rows were linearly independent. However, the expression simplifies to show that BX indeed projects onto the row space of X.

This shows that B is a right-inverse of X in the context of providing a solution to Xw = y when X has full row rank. Let me know if you have further questions!

Didn't explicitly state that this is the identity matrix but got to VV^T which is correct.

Prompt: solve question 4 The 5 interpretations of ridge regression

Response: (Perspective 1 is correct, Perspective to doesn't go into the details of the eigenvalue magnitudes, perspective 3 is fine and I followed up with a question on the steps, perspective 4

and 5 are incorrect, and it didn't answer the last few parts of the problem).

1. Perspective 1: Optimization Problem

Ridge regression is viewed as an unconstrained optimization problem with the objective function:
$\arg\min_w \|y - Xw\|_2^2 + \lambda\|w\|_2^2$
The addition of the $\lambda\|w\|_2^2$ term acts as a penalty on large weights, leading to the solution:
$w = (X^T X + \lambda I)^{-1} X^T y$

2. Perspective 2: "Hack" of Shifting the Singular Values

Using the SVD of $X = U\Sigma V^T$, the solution to ridge regression can be expressed as:
$w = (V\Sigma^T \Sigma V^T + \lambda I)^{-1} V\Sigma^T U^T y$
This simplifies to:
$w = V(\Sigma^T\Sigma + \lambda I)^{-1}\Sigma^T U^T y$
The singular values of $(X^T X + \lambda I)^{-1} X^T$ are shifted, and the impact of λ depends on the magnitude of σ_i relative to λ. When $\sigma_i << \lambda$, the singular values are significantly reduced, and when $\sigma_i >> \lambda$, the impact of λ is minimal.

3. Perspective 3: Maximum A Posteriori (MAP) Estimation

Ridge regression can be viewed as finding the MAP estimate for W given an observation $Y = y$, with a prior $W \sim N(0, I)$ and likelihood $Y = XW + \sqrt{\lambda}N$, where $N \sim N(0, 1)$. The MAP estimate for W is given by:
$\arg\min_w \|y - Xw\|_2^2 + \lambda\|w\|_2^2$
This is equivalent to the ridge regression optimization problem.

4. Perspective 4: Fake Data

Ridge regression can be interpreted as ordinary least squares on an augmented data set with fake data points. The augmented measurement vector $\hat{y}$ and data matrix $\hat{X}$ are:
$\hat{y} = \begin{bmatrix} y \\ 0_d \end{bmatrix}, \hat{X} = \begin{bmatrix} X \\ \sqrt{\lambda} I_d \end{bmatrix}$
The classical OLS optimization problem for $\hat{y}$ and $\hat{X}$ has the same minimizer as the original ridge regression problem.

5. Perspective 5: Fake Features

5. Perspective 5: Fake Features

Another interpretation involves constructing an augmented design matrix $\check{X} = [X \quad \sqrt{\lambda} I_n]$, leading to an underdetermined system. The min-norm solution to $\arg\min_\eta \|\eta\|_2^2$ subject to $\check{X}\eta = y$ yields the ridge regression solution for the first d coordinates of η^*.

These interpretations highlight the versatility and theoretical foundations of ridge regression, connecting it to optimization, linear algebra, Bayesian inference, and data augmentation.

explain how you got the answer for the map estimate

Deriving the MAP Estimate for Ridge Regression

The Maximum A Posteriori (MAP) estimate for ridge regression involves finding the mode of the posterior distribution of the weights given the observed data. Let's break it down step by step.

1. Prior Distribution

We assume a Gaussian prior distribution over the weights w:

$p(w) = \mathcal{N}(w|0, \frac{1}{\lambda} I) = \left(\frac{\lambda}{2\pi}\right)^{d/2} \exp\left(-\frac{\lambda}{2} w^T w\right)$

where λ is the precision (inverse variance) of the prior.

2. Likelihood

The likelihood of observing the data given the weights is also Gaussian:

$p(y|X, w) = \mathcal{N}(y|Xw, I) = \left(\frac{1}{2\pi}\right)^{n/2} \exp\left(-\frac{1}{2}(y - Xw)^T (y - Xw)\right)$

where n is the number of observations.

3. Posterior Distribution

Using Bayes' theorem, we can write the posterior distribution of the weights given the data as:

$p(w|X, y) \propto p(y|X, w) \cdot p(w)$

Taking the logarithm, we get:

$\log p(w|X, y) \propto -\frac{1}{2}(y - Xw)^T (y - Xw) - \frac{\lambda}{2} w^T w$

2. Likelihood

The likelihood of observing the data given the weights is also Gaussian:

$$p(y|X,w) = \mathcal{N}(y|Xw, I) = \left(\frac{1}{2\pi}\right)^{n/2} \exp\left(-\frac{1}{2}(y-Xw)^T(y-Xw)\right)$$

where n is the number of observations.

3. Posterior Distribution

Using Bayes' theorem, we can write the posterior distribution of the weights given the data as:

$$p(w|X,y) \propto p(y|X,w) \cdot p(w)$$

Taking the logarithm, we get:

$$\log p(w|X,y) \propto -\frac{1}{2}(y-Xw)^T(y-Xw) - \frac{\lambda}{2}w^Tw$$
$$= -\frac{1}{2}\left(y^Ty - 2y^TXw + w^TX^TXw\right) - \frac{\lambda}{2}w^Tw$$
$$= -\frac{1}{2}\left(w^T(X^TX + \lambda I)w - 2y^TXw\right) + \text{const}$$

4. MAP Estimate

To find the MAP estimate, we maximize the logarithm of the posterior distribution with respect to w. This is equivalent to minimizing the negative logarithm:

$$\hat{w}_{\text{MAP}} = \arg\min_w \frac{1}{2}\left(w^T(X^TX + \lambda I)w - 2y^TXw\right)$$
$$= \arg\min_w \frac{1}{2}\|y - Xw\|_2^2 + \frac{\lambda}{2}\|w\|_2^2$$

Taking the derivative with respect to w and setting it to zero, we get:

$$(X^TX + \lambda I)w = X^Ty$$
$$\hat{w}_{\text{MAP}} = (X^TX + \lambda I)^{-1}X^Ty$$

This is the same solution as ridge regression.

The MAP estimate provides a Bayesian interpretation of ridge regression, where the regularization term arises from the prior distribution over the weights.

Here it demonstrated some ability to show the steps needed to arrive at its answer, but I wasn't sure at all how to get it to give correct answers for other parts without actually giving it the answers.

Prompt: answer question 5 ReLU Elbow Update under SGD

Response: (it doesn't even answer part-by-part so I made another attempt – also unsuccessful).

ReLU Elbow Update under SGD

The "elbow" in the context of ReLU (Rectified Linear Unit) activation functions and Stochastic Gradient Descent (SGD) typically refers to a phenomenon where the loss landscape exhibits a particular structure that affects the optimization dynamics.

ReLU Activation Function

The ReLU function is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

Its derivative is:

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

SGD Update Rule

The SGD update rule for a parameter w with learning rate α is:

$$w_{t+1} = w_t - \alpha \cdot \nabla L$$

where ∇L is the gradient of the loss function with respect to w.

ReLU Elbow Update

For a ReLU activation function, the gradient of the loss with respect to the weights w will be zero for inputs that result in a ReLU output of 0 (i.e., when the input to ReLU is negative). This can lead to "dead" neurons during training.

When the input to a ReLU unit is close to zero, small changes in the weights can cause the unit to switch between being active (outputting a non-zero value) and being dead (outputting zero). This can create a sort of "elbow" in the loss landscape.

The update rule for weights connected to a ReLU unit under SGD would be:

$$w_{t+1} = w_t - \alpha \cdot \begin{cases} \nabla L \cdot x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

where x is the input to the ReLU unit.

Implications

The ReLU elbow update under SGD can lead to:

where x is the input to the ReLU unit.

Implications

The ReLU elbow update under SGD can lead to:

1. **Dead neurons**: If a ReLU unit outputs zero for most inputs, the corresponding weights may not be updated, leading to "dead" neurons.
2. **Non-differentiable points**: The ReLU function is non-differentiable at $x = 0$, which can affect the convergence of SGD.

To mitigate these issues, variants of ReLU, such as Leaky ReLU or Parametric ReLU, can be used. These variants allow a small gradient when the input is negative, which can help keep neurons active during training.

answer part-by-part 5. ReLU Elbow Update under SGD

I'll help you with the ReLU Elbow Update under SGD problem. Let's break it down part by part.

Step 1: Find the location of the 'elbow' e of the function $\phi(x)$

The 'elbow' of the ReLU function $\phi(x) = wx + b$ occurs when $wx + b = 0$. Solving for x, we get $x = -b/w$. This is the location of the elbow.

Step 2: Find the derivative of the loss w.r.t. $\phi(x)$

Given the loss function $\ell(x, y, \phi) = 1/2 \|\phi(x) - y\|^2_2$, we can find the derivative of the loss w.r.t. $\phi(x)$ as $d\ell/d\phi = \phi(x) - y$.

Step 3: Find the partial derivative of the loss w.r.t. w

To find $\partial\ell/\partial w$, we need to use the chain rule. Let's denote $\phi(x) = wx + b$. Then, $\partial\ell/\partial w = \partial\ell/\partial\phi * \partial\phi/\partial w = (\phi(x) - y) * x$.

Step 4: Find the partial derivative of the loss w.r.t. b

Similarly, $\partial\ell/\partial b = \partial\ell/\partial\phi * \partial\phi/\partial b = (\phi(x) - y) * 1$.

Step 5: Describe what happens to the slope and elbow of $\phi(x)$ when we perform gradient descent

We'll analyze each case separately.

Case (i): $\phi(x) = 0$

When $\phi(x) = 0$, the prediction is 0, and the target is y. The gradient update will try to move the function upwards. However, without knowing the specific values of w and b, we can't determine the exact change in the slope and elbow.

Case (ii): $w > 0$, $x > 0$, and $\phi(x) > 0$

In this case, the gradient update for w will be $\partial\ell/\partial w = (\phi(x) - y) * x$. Since $\phi(x) - y = 1$ (given), the update will be proportional to x. The slope of $\phi(x)$ will decrease since we're trying to pull the function downward.

Step 5: Describe what happens to the slope and elbow of $\phi(x)$ when we perform gradient descent

We'll analyze each case separately.

Case (i): $\phi(x) = 0$

When $\phi(x) = 0$, the prediction is 0, and the target is y. The gradient update will try to move the function upwards. However, without knowing the specific values of w and b, we can't determine the exact change in the slope and elbow.

Case (ii): $w > 0$, $x > 0$, and $\phi(x) > 0$

In this case, the gradient update for w will be $\partial \ell / \partial w = (\phi(x) - y) * x$. Since $\phi(x) - y = 1$ (given), the update will be proportional to x. The slope of $\phi(x)$ will decrease since we're trying to pull the function downward.

Case (iii): $w > 0$, $x < 0$, and $\phi(x) > 0$

The gradient update for w will be $\partial \ell / \partial w = (\phi(x) - y) * x$. Since $x < 0$, the update will be in the opposite direction of x. The slope of $\phi(x)$ will increase.

Case (iv): $w < 0$, $x > 0$, and $\phi(x) > 0$

The gradient update for w will be $\partial \ell / \partial w = (\phi(x) - y) * x$. Since $w < 0$, the update will try to make w more negative. The slope of $\phi(x)$ will decrease.

Step 6: Derive the new elbow location e' of the i'th elementwise ReLU activation after one stochastic gradient update with learning rate λ

Let's denote the original elbow location as $e = -b/w$. After one gradient update, the new weights and bias will be $w' = w - \lambda * \partial \ell / \partial w$ and $b' = b - \lambda * \partial \ell / \partial b$. The new elbow location e' will be $-b'/w' = -(b - \lambda * \partial \ell / \partial b) / (w - \lambda * \partial \ell / \partial w)$.

Case (iv): $w < 0$, $x > 0$, and $\phi(x) > 0$

The gradient update for w will be $\partial \ell / \partial w = (\phi(x) - y) * x$. Since $w < 0$, the update will try to make w more negative. The slope of $\phi(x)$ will decrease.

Step 6: Derive the new elbow location e' of the i'th elementwise ReLU activation after one stochastic gradient update with learning rate λ

Let's denote the original elbow location as $e = -b/w$. After one gradient update, the new weights and bias will be $w' = w - \lambda * \partial \ell / \partial w$ and $b' = b - \lambda * \partial \ell / \partial b$. The new elbow location e' will be $-b'/w' = -(b - \lambda * \partial \ell / \partial b) / (w - \lambda * \partial \ell / \partial w)$.

The final answer for each part is:

(i) $e = -b/w$
(ii) $d\ell/d\phi = \phi(x) - y$
(iii) $\partial \ell / \partial w = (\phi(x) - y) * x$
(iv) $\partial \ell / \partial b = \phi(x) - y$

Please let me know if you'd like me to elaborate on any of these steps or provide further clarification.

keep going answer the following parts

I'll continue with the rest of the problem.

Step 7: Calculate the new elbow location e' for the given cases

Let's calculate the new elbow location e' for each case.

Case (i): $\phi(x) = 0$

In this case, the gradient update for w is $\partial \ell / \partial w = (0 - y) * x$ and for b is $\partial \ell / \partial b = 0 - y$. The new weights and bias will be $w' = w - \lambda * (-y * x)$ and $b' = b - \lambda * (-y)$. The new elbow location e' will be $-b'/w' = -(b +$

Let's calculate the new elbow location e' for each case.

Case (i): $\phi(x) = 0$

In this case, the gradient update for w is $\partial \ell / \partial w = (0 - y) * x$ and for b is $\partial \ell / \partial b = 0 - y$. The new weights and bias will be $w' = w - \lambda * (-y * x)$ and $b' = b - \lambda * (-y)$. The new elbow location e' will be $-b'/w' = -(b + \lambda y) / (w + \lambda yx)$.

Case (ii): $w > 0$, $x > 0$, and $\phi(x) > 0$

Given $\phi(x) - y = 1$, the gradient update for w is $\partial \ell / \partial w = 1 * x$ and for b is $\partial \ell / \partial b = 1$. The new weights and bias will be $w' = w - \lambda * x$ and $b' = b - \lambda$. The new elbow location e' will be $-b'/w' = -(b - \lambda) / (w - \lambda x)$.

Case (iii): $w > 0$, $x < 0$, and $\phi(x) > 0$

Given $\phi(x) - y = 1$, the gradient update for w is $\partial \ell / \partial w = 1 * x$ and for b is $\partial \ell / \partial b = 1$. The new weights and bias will be $w' = w - \lambda * x$ and $b' = b - \lambda$. The new elbow location e' will be $-b'/w' = -(b - \lambda) / (w - \lambda x)$.

Case (iv): $w < 0$, $x > 0$, and $\phi(x) > 0$

Given $\phi(x) - y = 1$, the gradient update for w is $\partial \ell / \partial w = 1 * x$ and for b is $\partial \ell / \partial b = 1$. The new weights and bias will be $w' = w - \lambda * x$ and $b' = b - \lambda$. The new elbow location e' will be $-b'/w' = -(b - \lambda) / (w - \lambda x)$.

Step 8: Analyze the change in the elbow location for each case

Let's analyze the change in the elbow location for each case.

Case (i): $\phi(x) = 0$

The change in the elbow location is $e' - e = -(b + \lambda y) / (w + \lambda yx) + b/w$.

Case (ii): $w > 0$, $x > 0$, and $\phi(x) > 0$

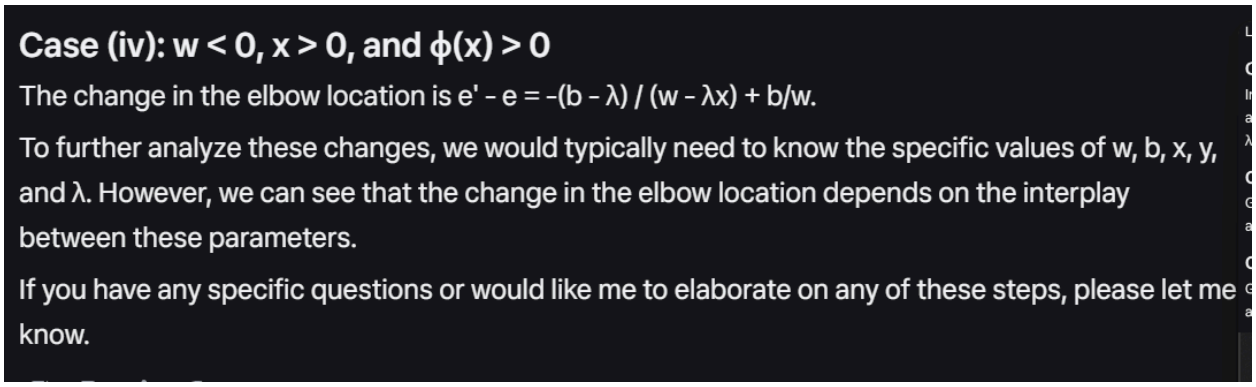
The change in the elbow location is $e' - e = -(b - \lambda) / (w - \lambda x) + b/w$.

Case (iii): $w > 0$, $x < 0$, and $\phi(x) > 0$

The change in the elbow location is $e' - e = -(b - \lambda) / (w - \lambda x) + b/w$.

Case (iv): $w < 0$, $x > 0$, and $\phi(x) > 0$

The change in the elbow location is $e' - e = -(b - \lambda) / (w - \lambda x) + b/w$



Seems like it struggles a lot answering part-by-part and is unable to handle multiple questions with subparts.